Poll / Quiz Widget

- **Problem:** Build an embeddable poll / quiz widget like a LinkedIn poll. It shows a question and its options. The user votes with one click and immediately sees result bars with percentages, the vote count and the time left. In a poll the user can undo their vote and vote again. In a quiz the widget reveals the correct answer and an explanation after the user answers, and the answer cannot be changed. Closed polls only show results. Votes survive a refresh, the UI updates optimistically and rolls back if the request fails.
- **Tech stack:** React (Vite, JavaScript, Tailwind CSS v4)
- **Time limit:** 60 minutes
- **Often asked at:** LinkedIn, Microsoft
- **Why it matters:** State, results bar

PART A: Thought process

General steps for any machine coding task

1. **Understand the requirements.** Restate the problem, split functional vs non-functional, ask clarifying questions, write down assumptions.
2. **List the interactions.** Every user action and its expected result. This is your checklist for building and testing.
3. **Design the architecture.** Name the components and their one-line jobs. Start small.
4. **Design the state.** What is the single source of truth? What can be derived?
5. **Design data structures and contracts.** Shape of the data, API calls, component props, reducer actions.
6. **Trace one data flow end to end.** Proves the design works before you type.
7. **Find the hard parts.** Solve them on paper first.
8. **Build in layers.** Simplest working version first, then one working layer at a time.
9. **Cover states and edge cases.** Loading, error, already voted, closed poll, zero votes, failed vote.
10. **Wrap up.** Trade-offs and what you would add with more time.

Time budget for 60 minutes

Phase	Time
Requirements + clarifying questions	5 min
Interactions, architecture, state design	6 min
Hard parts on paper (percent rounding, optimistic vote, hidden answer)	5 min
Build: mock API + load the poll + render options	10 min
Build: vote, results bars, percentages	10 min
Build: reducer with optimistic update + rollback, undo vote	9 min

Phase	Time
Build: quiz mode, closed poll, time left, persistence	8 min
Edge cases, accessibility, wrap-up	7 min

Tip: think out loud

Say every decision and its reason as you make it: "I store only the raw vote counts. Percentages and the total are derived, so they can never disagree with the counts." The interviewer grades how you think as much as the final code. If you get stuck, say what you are trying and what you would try next.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Load a poll by id from an API and show the **question** and its **options**.
- **Vote with one click.** Right after voting, show **result bars** with the percentage for each option, the user's choice marked, the **total votes** and the **time left**.
- **Poll mode:** the user can **undo** their vote and vote again while the poll is open.
- **Quiz mode:** after answering, show whether the answer was **correct**, highlight the correct option in green and a wrong pick in red, and show an **explanation**. The answer is final.
- **Closed poll** (end time passed): show results only, no voting.
- The vote **survives a refresh** (the server remembers it).

Non-functional requirements

Area	What it means here
Performance	Vote feels instant: optimistic update, no spinner blocking the UI. One small reducer, no extra libraries.
Scalability	Many widgets on one page (a feed). Each widget owns its own state, so one widget never re-renders another.
Error handling	Load failure shows a retry button. Vote failure rolls back the optimistic change and shows a message.
Accessibility	Options are real buttons with a group label. Results are a list with text percentages, the user's vote is announced, verdicts are in an `aria-live` region.
Reusability	`<PollWidget pollId="..." />` works anywhere. Poll and quiz are the same component driven by data.
Extensibility	Data shape ready for multi-select, live updates (WebSocket), or more question types.

Clarifying questions to ask the interviewer

1. Single choice only, or can a question allow several answers (multi-select)?

2. Should results be visible before voting, or only after voting / after the poll closes?
3. Can a user change or undo their vote? Is it the same for a quiz?
4. Is there a real API, or should I mock it? Does it return the user's previous vote?
5. Should counts update live when other people vote, or only on load and on my vote?
6. For a quiz, is it one question per widget, or a multi-step quiz with a final score?

Assumptions we will build with

- Single choice per question.
- Results are hidden until the user votes (avoids herd voting), but shown on a closed poll.
- Poll: undo is allowed while open. Quiz: one answer, locked.
- A mock async API with latency and an optional failure rate, backed by `localStorage` to act like a database. It returns the poll and the user's vote (`myVote`).
- The correct answer of a quiz is **not** sent to the client until the user has answered.
- No live updates; the server's counts replace ours after every vote.
- One question per widget. A multi-step quiz is a list of widgets (see the wrap-up).

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Widget mounts	Skeleton shown, poll fetched by id
Fetch fails	Error message + "Try again" button
Click "Try again"	Back to the skeleton, fetch again
Fetch succeeds, user has not voted, poll open	Question + one button per option
Fetch succeeds, user already voted	Result bars directly, own vote marked
Poll closed (end time passed)	Result bars, "Poll closed", no buttons
Click an option	Bars appear at once with the new counts (optimistic), "Saving…", other buttons disabled
Vote request succeeds	Counts replaced by the server's counts, "Saving…" gone
Vote request fails	Back to the options as before, error message in an alert
Double click / click another option fast	Only one request goes out
Poll: click "Undo vote"	Vote removed (optimistic), options shown again
Quiz: answer	"Checking your answer…", then green correct option, red wrong pick, verdict text + explanation
Quiz: answered	No undo button, server rejects a second answer
Refresh the page	Same vote and counts come back from the server
Zero votes on a poll	All bars at 0%, no division by zero

This table is the build and test checklist.

3. Design the architecture

Module	Responsibility
`App`	Demo page: renders a few widgets, a "flaky network" toggle and a reset button
`PollWidget`	Owns one poll's state: load, vote, undo, retry; picks options view vs results view
`ResultBar`	One result row: colored bar sized by percent, label, percent text
`pollReducer`	Pure state machine: load, optimistic vote, confirm, rollback
`poll.js` (utils)	Pure helpers: `toPercents`, `applyVote`, `totalVotes`, `timeLeft`
`pollApi`	Mock server: `fetchPoll`, `submitVote`, latency, failures, hides the answer key

```
App
 └─ PollWidget (pollId)            ← one per poll, independent state
      ├─ useReducer(pollReducer)   ← pure, testable logic
      ├─ pollApi.fetchPoll / submitVote
      ├─ options view: <button> × N
      └─ results view: <ResultBar> × N
```

Start with everything in `PollWidget`. Pull out `ResultBar` once the row markup grows (tone colors, screen reader text), and move the vote math to `utils` as soon as both the reducer and the mock server need `applyVote`.

---

4. Design the state / data model

State or field	Purpose
`status`	`'loading' | 'ready' | 'error'` for the fetch
`poll`	The poll as the server sent it (question, options with vote counts, end time, quiz fields after answering)
`myVote`	The option id this user picked, or `null`
`pending`	A vote request is in flight: disable buttons, show "Saving…"
`error`	Last error message (load or vote)
`snapshot`	`{ poll, myVote }` from before the optimistic update, used to roll back
`attempt` (separate `useState`)	Bumped by "Try again" to re-run the fetch effect
`inFlight` (ref)	Synchronous guard so a fast double click cannot send two requests

```
export const initialState = {
  status: 'loading', // 'loading' | 'ready' | 'error'
```

```
  poll: null,
  myVote: null,      // option id or null
  pending: false,    // a vote request is in flight
  error: null,
  snapshot: null,    // { poll, myVote } before the optimistic update
};
```

Senior note: derive, don't store.

- **Total votes** = sum of option votes. Storing it separately invites a bug where the total and the bars disagree.
- **Percentages** come from the counts on every render (`toPercents`).
- **showResults** = `myVote !== null || closed` . **closed** = `Date.now() > poll.endsAt` . **isQuiz** = `poll.type === 'quiz'` .
- **Ownership:** the server owns the truth (counts, `myVote` , the answer key). Each `PollWidget` owns a local copy plus UI flags. `App` owns only demo settings. No global store: widgets don't share state.

---

5. Design the data structures and contracts

Poll (as stored on the server)

```
{
  id: 'js-quiz',
  type: 'quiz',                      // 'poll' | 'quiz'
  question: 'What does typeof null return in JavaScript?',
  endsAt: 1793600000000,             // epoch ms; a number is easy to compare and store
  correctOptionId: 'object',         // quiz only, hidden until answered
  explanation: 'A bug from the first version of JavaScript...',
  options: [
    { id: 'null',   text: '"null"',   votes: 21 },
    { id: 'object', text: '"object"', votes: 34 },
  ],
}
```

Options have **stable string ids**, not array indexes: the vote is stored as an id, so reordering or adding options never moves a vote to the wrong option, and ids are safe React keys.

API contract (mocked)

Call	Input	Output	Errors
`fetchPoll(id)`	poll id	`{ poll, myVote }`	`Poll not found`
`submitVote(id, optionId)`	poll id, option id or `null` (retract)	`{ poll, myVote }` with fresh counts	network error, `This poll is closed.` , `You already answered.` (quiz), `Unknown option.`

Example response after answering a quiz:

```
{
  "poll": { "id": "js-quiz", "type": "quiz", "correctOptionId": "object",
            "explanation": "...", "options": [{ "id": "null", "text": "\"null\"", "votes": 22 }] },
  "myVote": "null"
}
```

The real endpoints would be `GET /polls/:id` and `PUT /polls/:id/vote { optionId }`. `PUT` (not `POST`) because setting your vote is idempotent: sending it twice leaves one vote.

Reducer actions

Action	Payload	Effect
`load_start`		Reset to loading
`load_ok`	`poll, myVote`	Ready
`load_fail`	`error`	Error state
`vote`	`optionId` (or `null`)	Save snapshot, apply vote locally, `pending = true`
`vote_ok`	`poll, myVote`	Replace with server data, clear snapshot
`vote_fail`	`error`	Restore snapshot, show error

Component contracts

- `<PollWidget pollId="framework" />`: one prop. Parent remounts it with `key={pollId}` when the id changes.
- `<ResultBar text percent votes tone mine />`: `tone` is `'neutral' | 'mine' | 'correct' | 'wrong'`.

---

6. Trace the data flow

User clicks "Vue" on an open poll:

```
click "Vue"
  │
  ▼
vote('vue')  ── inFlight.current? yes → ignore
  │ no
  ▼
dispatch({ type: 'vote', optionId: 'vue' })
  │  snapshot = { poll, myVote: null }
  │  poll = applyVote(poll, null, 'vue')   (Vue 12 → 13)
  │  myVote = 'vue', pending = true
  ▼
render: showResults = true → bars, Vue ✓, "Saving…"
  │  percents derived: toPercents([48, 17, 13, 3]) → [59, 21, 16, 4]
  ▼
```

```
await submitVote('framework', 'vue')   (600 ms, mock server)
  │
  ├── success → dispatch vote_ok(server poll, myVote)
  │              → counts replaced by the server's, "Saving…" gone
  │
  └── failure → dispatch vote_fail(error)
                 → poll + myVote restored from snapshot
                 → options view again + role="alert" message
  ▼
finally: inFlight.current = false
```

---

7. Important technical challenges

Challenge 1: Percentages that add up to exactly 100%

Problem. Three options with one vote each: `Math.round(33.33)` gives `33% + 33% + 33% = 99%`. With `[2, 2, 2, 1]` rounding gives `29 + 29 + 29 + 14 = 101%`. Users notice, and interviewers ask.

Solution. The **largest remainder method**: floor every share, then give the missing points one by one to the options with the biggest fractional parts. Zero total returns all zeros (no `NaN`).

```
export function toPercents(counts) {
  const total = counts.reduce((sum, c) => sum + c, 0);
  if (total === 0) return counts.map(() => 0);
  const raw = counts.map((c) => (c * 100) / total);
  const result = raw.map(Math.floor);
  let left = 100 - result.reduce((sum, p) => sum + p, 0);
  raw
    .map((r, i) => [r - result[i], i])
    .sort((a, b) => b[0] - a[0])
    .forEach(([, i]) => { if (left > 0) { result[i] += 1; left -= 1; } });
  return result;
}
```

Challenge 2: Optimistic vote with rollback

Problem. Waiting 600 ms for the server before showing results feels slow. But if we update first and the request fails, the UI shows a vote the server never saved. And between the click and the response, the user must not fire a second request.

Solution. The reducer saves a **snapshot** before applying the vote locally. On success the **server's data replaces ours** (other people may have voted meanwhile). On failure we **restore the snapshot**. A ref guards against a second request; the reducer also ignores a vote while `pending`.

```
case 'vote': {
  if (state.pending || state.myVote === action.optionId) return state;
```

```
  return {
    ...state, pending: true, error: null,
    snapshot: { poll: state.poll, myVote: state.myVote },
    poll: applyVote(state.poll, state.myVote, action.optionId),
    myVote: action.optionId,
  };
}
case 'vote_ok':
  return { ...state, pending: false, snapshot: null, poll: action.poll, myVote: action.myVote };
case 'vote_fail':
  return { ...state, ...state.snapshot, pending: false, snapshot: null, error: action.error };
```

`applyVote(poll, from, to)` handles all three cases with one function: first vote (`from = null`), change, and undo (`to = null`). The same function runs on the mock server, so client and server agree on the math.

Challenge 3: Quiz answer must not leak to the client

Problem. If the API sends `correctOptionId` with the question, anyone can open DevTools and see the answer, and the UI could accidentally render it. Also the client must not decide whether a quiz answer is final.

Solution. The server strips `correctOptionId` and `explanation` until the user has answered, and rejects a second answer. The client shows "Checking your answer…" while the optimistic vote waits; the colors and verdict only appear once the server reveals the key.

```
function toClient(id) {
  const poll = db[id];
  const myVote = myVotes[id] ?? null;
  if (myVote) return { poll, myVote };
  const { correctOptionId, explanation, ...hidden } = poll;
  return { poll: hidden, myVote };
}
```

```
function toneFor(option, poll, myVote) {
  const mine = option.id === myVote;
  if (poll.type !== 'quiz' || !poll.correctOptionId) return mine ? 'mine' : 'neutral';
  if (option.id === poll.correctOptionId) return 'correct';
  return mine ? 'wrong' : 'neutral';
}
```

Challenge 4: Async loading without stale responses

Problem. The widget unmounts (user scrolls a virtualized feed) or `pollId` changes before the fetch resolves. In StrictMode the effect runs twice in dev. A late response for the old poll would overwrite the new one.

Solution. An `ignore` flag in the effect cleanup drops stale responses. Retry is a counter in the effect's dependencies, so "Try again" re-runs the same effect instead of duplicating the fetch code.

```
useEffect(() => {
  let ignore = false;
  fetchPoll(pollId).then(
    (res) => !ignore && dispatch({ type: 'load_ok', ...res }),
    (err) => !ignore && dispatch({ type: 'load_fail', error: err.message })
  );
  return () => { ignore = true; };
}, [pollId, attempt]);
```

---

8. Implement in the correct order

#	Step	Working result
1	Vite + Tailwind, `App` renders one hard-coded `PollWidget`	Page shows a static question
2	Mock API with seed data and latency	`fetchPoll` returns data after 600 ms
3	Load in an effect, loading skeleton, error + retry	Widget shows options
4	Options as buttons, `vote()` calls `submitVote` and stores the result	Clicking votes (slowly, no optimism yet)
5	Results view: `ResultBar`, `totalVotes`, `toPercents`	Bars with percents that sum to 100
6	Move to `useReducer`; optimistic `vote`, `vote_ok`, `vote_fail`	Instant bars, rollback on failure
7	`inFlight` ref + disabled buttons + "Saving…"	No double requests
8	Undo vote (`vote(null)`), closed poll, `timeLeft`	Full poll behavior
9	Quiz mode: hidden answer key, tones, verdict, explanation	Full quiz behavior
10	Persistence in the mock server (`localStorage`), reset button, flaky toggle	Survives refresh, failures can be demoed
11	Accessibility pass: group label, sr-only text, `aria-live`, `aria-busy`, focus ring	Screen reader friendly
12	Node self-check of the pure logic	`node check.mjs` passes

---

9. Cover all states and edge cases

State / edge case	What the user sees	How
Initial / loading	Pulsing skeleton, `aria-busy`	`status === 'loading'`
Load error	Message + "Try again"	`status === 'error'`, `attempt` counter
Ready, not voted, open	Option buttons	`showResults === false`

State / edge case	What the user sees	How
Already voted (after refresh)	Results directly	Server returns `myVote`
Vote pending	Bars, "Saving…", buttons disabled	`pending`, `inFlight` ref
Vote failed	Options again + alert	Snapshot rollback
Undo vote	Options again, counts minus one	`applyVote(poll, myVote, null)`
Closed poll	Results, "Poll closed", no undo	`Date.now() > endsAt`
Vote on a poll that closed after loading	Rollback + "This poll is closed."	Server check
Quiz pending	"Checking your answer…"	`myVote` set, no `correctOptionId` yet
Quiz correct / wrong	Green / red bars, verdict, explanation	`toneFor`
Quiz answered twice (other tab)	Error, rollback	Server rejects
Zero votes	All bars 0%, "0 votes"	`toPercents` guard
1 vote	"1 vote" (singular)	Pluralisation
Rounding	Always 100% total	Largest remainder
Unknown poll id	Load error	Server throws
Unmount during fetch	Nothing breaks	`ignore` flag
`localStorage` blocked / full	Works in memory	`try/catch` in the mock

10. Final structure

```
_81_poll_quiz_widget/
├── index.html
├── vite.config.js
├── package.json
├── check.mjs                 # node self-check of the pure logic + mock API
└── src/
    ├── main.jsx
    ├── index.css
    ├── App.jsx               # demo page
    ├── api/
    │   └── pollApi.js        # mock server
    ├── utils/
    │   ├── poll.js           # toPercents, applyVote, totalVotes, timeLeft
    │   └── pollReducer.js    # optimistic state machine
    └── components/
        ├── PollWidget.jsx
        └── ResultBar.jsx
```

11. Full working code

Setup: `npm create vite@latest` (React, JavaScript), then `npm i -D tailwindcss @tailwindcss/vite`.

`vite.config.js`

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react-swc';
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

`src/index.css`

```
@import "tailwindcss";

body {
  background: var(--color-slate-50);
}
```

`src/main.jsx`

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './index.css';
import App from './App';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

`src/utils/poll.js`

```
// Largest remainder method: floor every share, then give the leftover points
// to the biggest fractions so the bars always add up to exactly 100%.
export function toPercents(counts) {
  const total = counts.reduce((sum, c) => sum + c, 0);
  if (total === 0) return counts.map(() => 0);
  const raw = counts.map((c) => (c * 100) / total);
  const result = raw.map(Math.floor);
  let left = 100 - result.reduce((sum, p) => sum + p, 0);
```

```
  raw
    .map((r, i) => [r - result[i], i])
    .sort((a, b) => b[0] - a[0])
    .forEach(([, i]) => {
      if (left > 0) {
        result[i] += 1;
        left -= 1;
      }
    });
  return result;
}

// Moves one vote from `from` to `to` (either can be null: first vote / retract).
export function applyVote(poll, from, to) {
  if (from === to) return poll;
  const options = poll.options.map((o) => {
    if (o.id === from) return { ...o, votes: o.votes - 1 };
    if (o.id === to) return { ...o, votes: o.votes + 1 };
    return o;
  });
  return { ...poll, options };
}

export function totalVotes(poll) {
  return poll.options.reduce((sum, o) => sum + o.votes, 0);
}

export function timeLeft(endsAt, now = Date.now()) {
  const ms = endsAt - now;
  if (ms <= 0) return 'Poll closed';
  const hours = Math.floor(ms / 3_600_000);
  if (hours >= 24) return `${Math.floor(hours / 24)}d left`;
  if (hours >= 1) return `${hours}h left`;
  return `${Math.max(1, Math.ceil(ms / 60_000))}m left`;
}
```

src/utils/pollReducer.js

```
import { applyVote } from './poll.js';

export const initialState = {
  status: 'loading', // 'loading' | 'ready' | 'error'
  poll: null,
  myVote: null, // option id or null
  pending: false, // a vote request is in flight
  error: null,
  snapshot: null, // { poll, myVote } before the optimistic update, for rollback
```

```
};

export function pollReducer(state, action) {
  switch (action.type) {
    case 'load_start':
      return { ...initialState };
    case 'load_ok':
      return { ...state, status: 'ready', poll: action.poll, myVote: action.myVote, error: null };
    case 'load_fail':
      return { ...state, status: 'error', error: action.error };
    case 'vote': {
      if (state.pending || state.myVote === action.optionId) return state;
      return {
        ...state,
        pending: true,
        error: null,
        snapshot: { poll: state.poll, myVote: state.myVote },
        poll: applyVote(state.poll, state.myVote, action.optionId),
        myVote: action.optionId,
      };
    }
    case 'vote_ok':
      // The server's counts win: other people may have voted meanwhile.
      return { ...state, pending: false, snapshot: null, poll: action.poll, myVote: action.myVote };
    case 'vote_fail':
      return { ...state, ...state.snapshot, pending: false, snapshot: null, error: action.error };
    default:
      return state;
  }
}
```

src/api/pollApi.js

```
import { applyVote } from '../utils/poll.js';

// Mock backend: async, slow, can fail, and keeps data in localStorage like a real server would keep it in a DB.
const DB_KEY = 'poll-db-v1';
const VOTES_KEY = 'poll-my-votes-v1';
const DAY = 86_400_000;

export const config = { latency: 600, failRate: 0 };

function seed() {
  const now = Date.now();
  return {
    framework: {
```

```
      id: 'framework',
      type: 'poll',
      question: 'Which frontend framework do you use most at work?',
      endsAt: now + 3 * DAY,
      options: [
        { id: 'react', text: 'React', votes: 48 },
        { id: 'angular', text: 'Angular', votes: 17 },
        { id: 'vue', text: 'Vue', votes: 12 },
        { id: 'svelte', text: 'Svelte', votes: 3 },
      ],
    },
    'js-quiz': {
      id: 'js-quiz',
      type: 'quiz',
      question: 'What does typeof null return in JavaScript?',
      endsAt: now + 7 * DAY,
      correctOptionId: 'object',
      explanation: 'A bug from the first version of JavaScript, kept for backward compatibility.',
      options: [
        { id: 'null', text: '"null"', votes: 21 },
        { id: 'object', text: '"object"', votes: 34 },
        { id: 'undefined', text: '"undefined"', votes: 9 },
        { id: 'number', text: '"number"', votes: 2 },
      ],
    },
    remote: {
      id: 'remote',
      type: 'poll',
      question: 'Would you take a pay cut for fully remote work?',
      endsAt: now - DAY,
      options: [
        { id: 'yes', text: 'Yes', votes: 64 },
        { id: 'no', text: 'No', votes: 51 },
        { id: 'depends', text: 'Depends on the cut', votes: 30 },
      ],
    },
  };
}

function read(key, fallback) {
  try {
    return JSON.parse(localStorage.getItem(key)) ?? fallback;
  } catch {
    return fallback; // no storage (private mode, Node) or corrupt JSON
  }
}
```

```
function write(key, value) {
  try {
    localStorage.setItem(key, JSON.stringify(value));
  } catch {
    // storage blocked or full: keep working in memory
  }
}

let db = read(DB_KEY, null) ?? seed();
let myVotes = read(VOTES_KEY, {});

const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

// The answer key never reaches the client until the user has answered.
function toClient(id) {
  const poll = db[id];
  const myVote = myVotes[id] ?? null;
  if (myVote) return { poll, myVote };
  // eslint-disable-next-line no-unused-vars
  const { correctOptionId, explanation, ...hidden } = poll;
  return { poll: hidden, myVote };
}

export async function fetchPoll(id) {
  await wait(config.latency);
  if (!db[id]) throw new Error('Poll not found');
  return toClient(id);
}

// optionId = null retracts the vote.
export async function submitVote(id, optionId) {
  await wait(config.latency);
  if (Math.random() < config.failRate) throw new Error('Network error. Your vote was not saved.');
  const poll = db[id];
  if (!poll) throw new Error('Poll not found');
  if (Date.now() > poll.endsAt) throw new Error('This poll is closed.');
  if (poll.type === 'quiz' && myVotes[id]) throw new Error('You already answered.');
  if (optionId !== null && !poll.options.some((o) => o.id === optionId)) throw new Error('Unknown
option.');
  db = { ...db, [id]: applyVote(poll, myVotes[id] ?? null, optionId) };
  myVotes = { ...myVotes, [id]: optionId };
  write(DB_KEY, db);
  write(VOTES_KEY, myVotes);
  return toClient(id);
}

export function resetDemo() {
```

```
  db = seed();
  myVotes = {};
  write(DB_KEY, db);
  write(VOTES_KEY, myVotes);
}
```

src/components/ResultBar.jsx

```
const TONES = {
  neutral: 'bg-slate-200',
  mine: 'bg-blue-200',
  correct: 'bg-green-200',
  wrong: 'bg-red-200',
};

export default function ResultBar({ text, percent, votes, tone, mine }) {
  return (
    <li className="relative overflow-hidden rounded-md border border-slate-200">
      <div
        aria-hidden="true"
        className={`absolute inset-y-0 left-0 ${TONES[tone]} transition-[width] duration-500`}
        style={{ width: `${percent}%` }}
      />
      <div className="relative flex items-center justify-between gap-3 px-3 py-2 text-sm">
        <span className={mine ? 'font-semibold' : undefined}>
          {text}
          {mine && <span aria-hidden="true"> ✓</span>}
          {mine && <span className="sr-only"> (your vote)</span>}
          {tone === 'correct' && <span className="sr-only"> (correct answer)</span>}
        </span>
        <span className="tabular-nums" title={`${votes} votes`}>
          {percent}%
        </span>
      </div>
    </li>
  );
}
```

src/components/PollWidget.jsx

```
import { useEffect, useId, useReducer, useRef, useState } from 'react';
import { fetchPoll, submitVote } from '../api/pollApi';
import { initialState, pollReducer } from '../utils/pollReducer';
import { timeLeft, toPercents, totalVotes } from '../utils/poll';
import ResultBar from './ResultBar';
```

```
function Card({ children, busy }) {
  return (
    <section aria-busy={busy} className="rounded-xl border border-slate-200 bg-white p-4 shadow-sm">
      {children}
    </section>
  );
}

function toneFor(option, poll, myVote) {
  const mine = option.id === myVote;
  if (poll.type !== 'quiz' || !poll.correctOptionId) return mine ? 'mine' : 'neutral';
  if (option.id === poll.correctOptionId) return 'correct';
  return mine ? 'wrong' : 'neutral';
}

export default function PollWidget({ pollId }) {
  const [state, dispatch] = useReducer(pollReducer, initialState);
  const [attempt, setAttempt] = useState(0);
  const inFlight = useRef(false); // blocks a second request before React re-renders the disabled
buttons
  const headingId = useId();

  useEffect(() => {
    let ignore = false; // drop the response if the widget unmounted or pollId changed
    fetchPoll(pollId).then(
      (res) => !ignore && dispatch({ type: 'load_ok', ...res }),
      (err) => !ignore && dispatch({ type: 'load_fail', error: err.message })
    );
    return () => {
      ignore = true;
    };
  }, [pollId, attempt]);

  async function vote(optionId) {
    if (inFlight.current || optionId === state.myVote) return;
    inFlight.current = true;
    dispatch({ type: 'vote', optionId });
    try {
      dispatch({ type: 'vote_ok', ...(await submitVote(pollId, optionId)) });
    } catch (err) {
      dispatch({ type: 'vote_fail', error: err.message });
    } finally {
      inFlight.current = false;
    }
  }

  function retry() {
```

```
    dispatch({ type: 'load_start' });
    setAttempt((a) => a + 1);
  }

  const { status, poll, myVote, pending, error } = state;

  if (status === 'loading') {
    return (
      <Card busy>
        <p className="sr-only">Loading poll…</p>
        <div aria-hidden="true" className="animate-pulse space-y-3">
          <div className="h-5 w-3/4 rounded bg-slate-200" />
          {[0, 1, 2].map((i) => (
            <div key={i} className="h-10 rounded-full bg-slate-100" />
          ))}
        </div>
      </Card>
    );
  }

  if (status === 'error') {
    return (
      <Card>
        <p role="alert" className="text-sm text-red-700">
          Could not load the poll: {error}
        </p>
        <button type="button" onClick={retry} className="mt-3 rounded-md border px-3 py-1 text-sm
hover:bg-slate-50">
          Try again
        </button>
      </Card>
    );
  }

  const isQuiz = poll.type === 'quiz';
  const closed = Date.now() > poll.endsAt;
  const showResults = myVote !== null || closed;
  const total = totalVotes(poll);
  const percents = toPercents(poll.options.map((o) => o.votes));
  const correctText = poll.options.find((o) => o.id === poll.correctOptionId)?.text;

  return (
    <Card busy={pending}>
      <p className="text-xs font-semibold uppercase tracking-wide text-slate-500">{isQuiz ? 'Quiz' :
'Poll'}</p>
      <h2 id={headingId} className="mt-1 mb-3 font-semibold text-slate-900">
        {poll.question}
```

```
      </h2>

      {showResults ? (
        <ul aria-labelledby={headingId} className="space-y-2">
          {poll.options.map((o, i) => (
            <ResultBar
              key={o.id}
              text={o.text}
              votes={o.votes}
              percent={percents[i]}
              mine={o.id === myVote}
              tone={toneFor(o, poll, myVote)}
            />
          ))}
        </ul>
      ) : (
        <div role="group" aria-labelledby={headingId} className="space-y-2">
          {poll.options.map((o) => (
            <button
              key={o.id}
              type="button"
              onClick={() => vote(o.id)}
              disabled={pending}
              className="w-full rounded-full border-2 border-blue-600 px-4 py-2 text-sm font-semibold
text-blue-700 hover:bg-blue-50 focus-visible:outline-2 focus-visible:outline-offset-2 focus-
visible:outline-blue-600 disabled:opacity-50"
            >
              {o.text}
            </button>
          ))}
        </div>
      )}

      <div aria-live="polite" className="mt-3 space-y-1 text-sm">
        {isQuiz && myVote !== null && !poll.correctOptionId && <p className="text-slate-600">Checking
your answer…</p>}
        {isQuiz && poll.correctOptionId && (
          <>
            <p className={myVote === poll.correctOptionId ? 'font-semibold text-green-700' : 'font-
semibold text-red-700'}>
              {myVote === poll.correctOptionId ? 'Correct!' : `Not quite. The answer is
${correctText}.`}
            </p>
            <p className="text-slate-600">{poll.explanation}</p>
          </>
        )}
        {error && (
```

```
          <p role="alert" className="text-red-700">
            {error}
          </p>
        )}
      </div>

      <div className="mt-3 flex items-center gap-2 text-xs text-slate-500">
        <span>
          {total} {total === 1 ? 'vote' : 'votes'} · {timeLeft(poll.endsAt)}
        </span>
        {pending && <span>· Saving…</span>}
        {!isQuiz && !closed && myVote !== null && !pending && (
          <button type="button" onClick={() => vote(null)} className="ml-auto font-semibold text-blue-
700 hover:underline">
            Undo vote
          </button>
        )}
      </div>
    </Card>
  );
}
```

src/App.jsx

```
import { useState } from 'react';
import PollWidget from './components/PollWidget';
import { config, resetDemo } from './api/pollApi';

const POLL_IDS = ['framework', 'js-quiz', 'remote'];

export default function App() {
  const [flaky, setFlaky] = useState(false);

  function toggleFlaky(e) {
    config.failRate = e.target.checked ? 0.5 : 0;
    setFlaky(e.target.checked);
  }

  function reset() {
    resetDemo();
    window.location.reload();
  }

  return (
    <main className="mx-auto min-h-screen max-w-xl space-y-4 p-4">
      <h1 className="text-2xl font-bold text-slate-900">Poll &amp; Quiz Widget</h1>
      <div className="flex flex-wrap items-center gap-4 text-sm text-slate-700">
```

```
        <label className="flex items-center gap-2">
          <input type="checkbox" checked={flaky} onChange={toggleFlaky} />
          Simulate flaky network (50% of votes fail)
        </label>
        <button type="button" onClick={reset} className="rounded-md border bg-white px-3 py-1 hover:bg-
slate-100">
          Reset demo data
        </button>
      </div>
      {POLL_IDS.map((id) => (
        <PollWidget key={id} pollId={id} />
      ))}
    </main>
  );
}
```

`check.mjs` (run with `node check.mjs`)

```
import assert from 'node:assert/strict';
import { applyVote, timeLeft, toPercents, totalVotes } from './src/utils/poll.js';
import { initialState, pollReducer } from './src/utils/pollReducer.js';
import { config, fetchPoll, submitVote } from './src/api/pollApi.js';

// percentages always sum to 100
assert.deepEqual(toPercents([1, 1, 1]), [34, 33, 33]);
assert.deepEqual(toPercents([0, 0]), [0, 0]);
assert.deepEqual(toPercents([5]), [100]);
for (const counts of [[48, 17, 12, 3], [2, 2, 2, 1], [1, 2, 3, 4, 5, 6, 7]]) {
  assert.equal(toPercents(counts).reduce((a, b) => a + b), 100);
}

// moving a vote
const poll = { id: 'p', options: [{ id: 'a', votes: 2 }, { id: 'b', votes: 1 }] };
assert.deepEqual(applyVote(poll, null, 'a').options.map((o) => o.votes), [3, 1]);
assert.deepEqual(applyVote(poll, 'a', 'b').options.map((o) => o.votes), [1, 2]);
assert.deepEqual(applyVote(poll, 'a', null).options.map((o) => o.votes), [1, 1]);
assert.equal(applyVote(poll, 'a', 'a'), poll);
assert.equal(totalVotes(poll), 3);

// optimistic vote, then rollback on failure
let s = pollReducer(initialState, { type: 'load_ok', poll, myVote: null });
s = pollReducer(s, { type: 'vote', optionId: 'b' });
assert.equal(s.pending, true);
assert.equal(s.myVote, 'b');
assert.equal(s.poll.options[1].votes, 2);
assert.equal(pollReducer(s, { type: 'vote', optionId: 'a' }), s); // ignored while pending
const failed = pollReducer(s, { type: 'vote_fail', error: 'boom' });
```

```
assert.equal(failed.poll, poll);
assert.equal(failed.myVote, null);
assert.equal(failed.pending, false);
assert.equal(failed.error, 'boom');

// time left
const now = 1_000_000_000;
assert.equal(timeLeft(now - 1, now), 'Poll closed');
assert.equal(timeLeft(now + 3 * 86_400_000, now), '3d left');
assert.equal(timeLeft(now + 5 * 3_600_000, now), '5h left');
assert.equal(timeLeft(now + 30_000, now), '1m left');

// mock API: answer key hidden until answered, quiz locked after, closed poll rejects
config.latency = 0;
const quiz = await fetchPoll('js-quiz');
assert.equal(quiz.poll.correctOptionId, undefined);
const answered = await submitVote('js-quiz', 'null');
assert.equal(answered.poll.correctOptionId, 'object');
assert.equal(answered.myVote, 'null');
await assert.rejects(submitVote('js-quiz', 'object'), /already answered/);
await assert.rejects(submitVote('remote', 'yes'), /closed/);
await assert.rejects(fetchPoll('nope'), /not found/);
const voted = await submitVote('framework', 'vue');
assert.equal(voted.poll.options.find((o) => o.id === 'vue').votes, 13);
const retracted = await submitVote('framework', null);
assert.equal(retracted.myVote, null);
assert.equal(retracted.poll.options.find((o) => o.id === 'vue').votes, 12);

console.log('all checks passed');
```

---

12. Wrap-up

Edge cases handled

- Percentages always sum to 100, zero votes gives 0% (no `NaN`).
- Optimistic vote with rollback; double clicks send one request.
- Undo vote in polls; quiz answers are final and checked on the server.
- Quiz answer key never reaches the client before answering.
- Closed polls show results only; a poll that closes after loading is rejected by the server and rolled back.
- Stale fetch responses are ignored; storage failures don't crash the widget.

Trade-offs to mention

Decision	Why	Cost
One-click vote (no "Submit")	LinkedIn-style, fastest	A misclick votes; undo covers polls, not quizzes

Decision	Why	Cost
Optimistic update	Feels instant	Must handle rollback and server reconciliation
`useReducer` per widget, no global store	Widgets are independent, logic is pure and testable	Two widgets of the same poll on one page don't sync
Results hidden until voting	Avoids herd voting	Users who don't vote never see results
`Date.now()` checked at render, no ticking timer	Simple	"2d left" doesn't count down live; "closed" appears on the next render
Server counts replace ours after a vote	Always correct	Bars may jump a little if others voted meanwhile

What to add with more time

- **Tests:** React Testing Library for click → bars → rollback, with a mocked API.
- **Live results:** WebSocket / SSE pushes new counts; merge them unless a vote is pending.
- **Multi-select** questions (`myVotes: string[]` , total voters stored by the server).
- **Multi-step quiz:** a `Quiz` parent with an index, score and a summary screen, reusing `PollWidget` per question.
- **Animations:** bars grow from 0 on reveal (`@starting-style` or `transform: scaleX`).
- **Live countdown** with one shared interval for all widgets.
- **Data layer:** TanStack Query for caching, retries and optimistic mutations.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why hide results until the user votes?

Seeing the current leader biases people toward it (herd effect), and it is how LinkedIn works. It is one boolean (`showResults`), so if the interviewer wants results always visible, it changes to `true` without touching anything else.

Q2. What if they want multi-select?

`myVote` becomes an array, the UI becomes checkboxes plus a "Vote" button, and `applyVote` diffs two sets. The total can no longer be derived from the option counts (one person adds several votes), so the server must return `totalVoters` and percentages are per voter, so they no longer sum to 100.

State design

Q3. Why `useReducer` and not several `useState` calls?

A vote changes `poll` , `myVote` , `pending` , `error` and `snapshot` together. With separate setters it is easy to update four and forget the fifth. The reducer makes every transition one atomic, pure function that I can test in Node without React.

Q4. Why don't you store `totalVotes`?

It is the sum of option votes. A stored copy can drift (update the option, forget the total) and the bars stop matching the number. Deriving it costs a loop over 2-5 options.

Q5. Why is `inFlight` a ref when you already have `pending` in state?

State updates are applied on the next render, so two events before that render would both see `pending === false` in the closure. A ref changes synchronously. It also stays out of rendering because nothing on screen depends on it.

Q6. Why `key={pollId}` on the widget?

If the `pollId` prop changes, `key` makes React mount a fresh widget with fresh state, instead of showing the old poll's results until the new fetch arrives. It is cheaper than writing reset logic.

Challenge 1: percentages

Q7. Why not `Math.round` each percentage?

`[1, 1, 1]` gives 99% and `[2, 2, 2, 1]` gives 101%. The largest remainder method floors all values and gives the leftover points to the biggest fractions, so the sum is always exactly 100.

Q8. Two options are tied, which one gets the extra point?

The sort is stable, so ties keep their original order and the first option wins. It is deterministic, so the bars don't flicker between renders. If the business wants, show one decimal instead.

Challenge 2: optimistic updates

Q9. Why replace local counts with the server's after success?

Other people voted while my request was in flight. The server's counts are the truth. Incrementing locally again would double count my own vote.

Q10. What if the user votes, then quickly clicks "Undo" before the first request returns?

The undo button is hidden while `pending`, the ref blocks a second request, and the reducer ignores `vote` while pending. If you wanted to allow it, you would queue the latest intent or send requests with a version number and let the server keep the last one.

Q11. What if the API is very slow?

The UI already shows results, so the user is not blocked. Add a timeout with `AbortController` (e.g. 10 s) that dispatches `vote_fail`, and a "Retry" next to the error. The real fix is server side; the client just must never hang in "Saving…" forever.

Q12. What if the request succeeded on the server but the response was lost?

The client rolls back and shows an error, but the vote exists. Because the endpoint is idempotent (`PUT` my vote), retrying is safe and the next `fetchPoll` returns the true `myVote`. That is why `myVote` comes from the server, not from local storage.

Challenge 3: quiz security

Q13. Can't the user still cheat?

They can answer, see the key, and tell friends, like any quiz. What we prevent is reading the answer before answering and changing the answer afterwards. Both checks are on the server; the client only reflects them.

Q14. Why does the client not compute "correct / wrong" itself?

It does not have the key until the server says so. That keeps one source of truth and lets the server add rules (time limits, scoring) without a client change.

Challenge 4: async loading

Q15. Why an `ignore` flag and not `AbortController`?

Both work. `ignore` is one line and works with any promise. `AbortController` also cancels the network request, which matters for big payloads or when a real `fetch` is used. I would switch to it with a real API.

Q16. Why does the effect fire twice in development?

React StrictMode mounts, unmounts and mounts again to surface missing cleanups. With the `ignore` flag the first response is dropped, so the UI is correct. In production it fires once.

Performance

Q17. What if a feed has 1,000 polls?

Each widget is independent, so a vote re-renders only its own card. For the list itself, virtualize it (render only visible cards) and fetch lazily with an `IntersectionObserver`, or batch fetches (`GET /polls?ids=...`).

Q18. What if a poll has 1 million votes?

The counts are just numbers, so nothing changes on the client. The server must not count rows on every read: keep a counter per option, update it atomically, and cache reads. Format large numbers (`1.2M votes`) with `Intl.NumberFormat`.

Q19. How would you show live results?

Subscribe to a WebSocket / SSE channel per visible poll and dispatch `counts_updated`. While a vote is pending, ignore pushes (or apply them to the snapshot) so they don't overwrite the optimistic state. Throttle updates to about one per second so bars don't jitter.

Accessibility

Q20. Why buttons and not radio inputs?

Native radios change selection on arrow keys, which would cast a vote on every arrow press. Buttons vote only on Enter / Space / click. The group gets `aria-labelledby` the question, so a screen reader reads the question with the options.

Q21. How does a screen reader user know the result?

Results are a list where each item has the option text and the percent as real text, plus sr-only "(your vote)" and "(correct answer)". The verdict and errors are in an `aria-live` region / `role="alert"`, so they are announced without moving focus.

Trade-offs

Q22. Why not keep the user's vote in `localStorage` on the client?

It breaks across devices and browsers, and the user can edit it. The server is the source of truth and returns `myVote`. In this demo, `localStorage` is used only inside the mock to play the role of the database.

Q23. Why not TanStack Query or Redux?

For one widget, a reducer and an effect are about 40 lines and show the interviewer I understand optimistic updates. In a real app with many polls and caching needs, TanStack Query's `useMutation` with `onMutate` / `onError` gives the same snapshot-and-rollback pattern for free.